Date:- 24/7/26
Exp no:- 1

Network components used in Linux

Aim :-

To study various network commands used in Linux and windows.

Commands :-

arp - a → shows Ip address of the computer along with the ip and MAC Address.

hostname → displays name of the computer

ipconfig → displays configuration in the pc

nbstat → Helps to solve problems with NetBIOS

nslookup → displays detail DNS

pathping → Traces the route to the destination address then launches a 25 second test of each router along the way.

ping → Best way of test connectivity between two nodes

```
# ping hostname
# ping ip address
# ping
```

```
If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*            .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
              destination^      ^mask      ^gateway     metric^    ^
                                                        Interface^
      If IF is not given, it tries to find the best interface for a given
      gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

      CHANGE is used to modify gateway and/or metric only.

    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

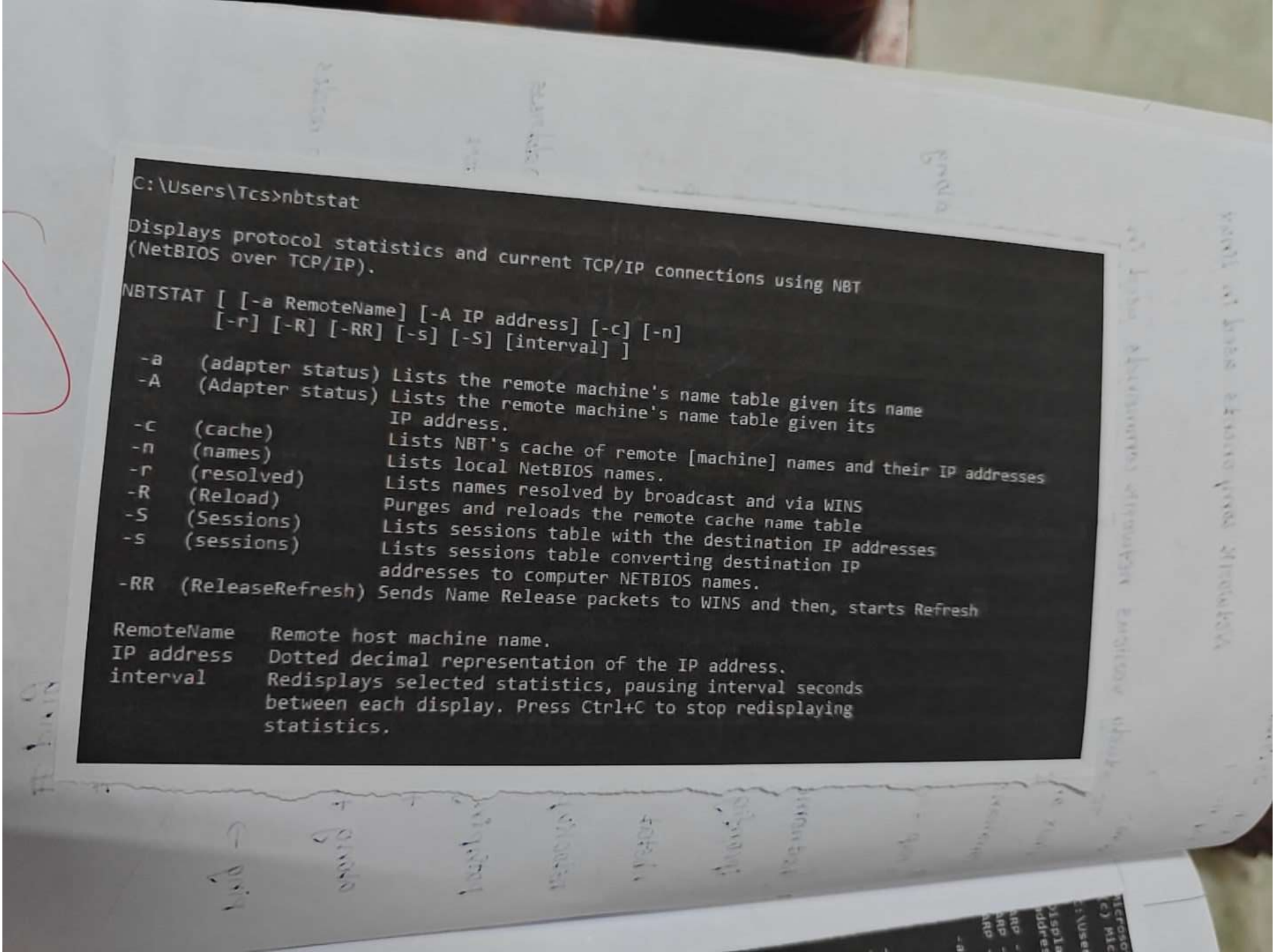
C:\Users\Tcs>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.

Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.
```



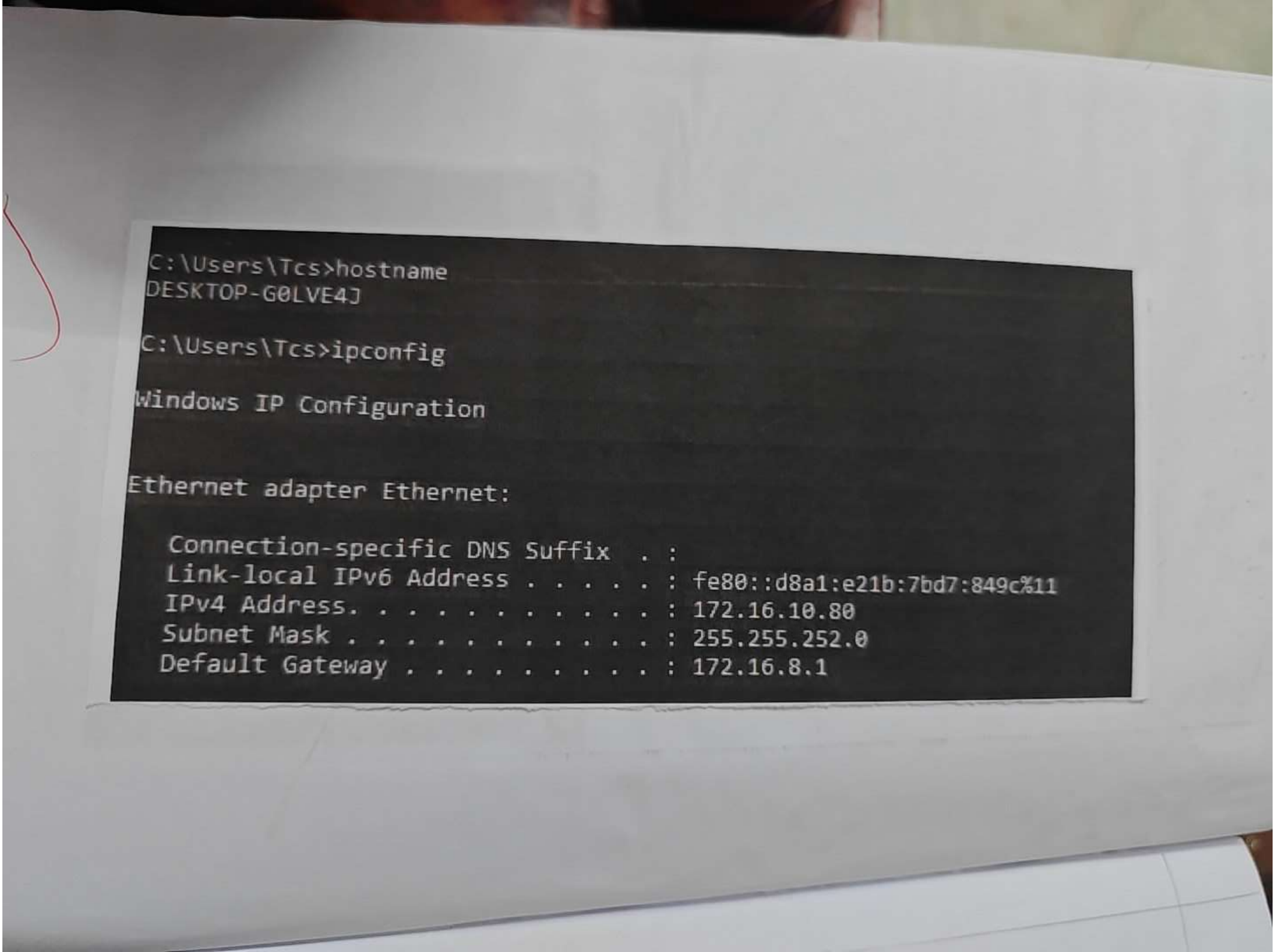
```
C:\Users\Tcs>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```


C:\Users\Tcs>hostname
DESKTOP-G0LVE4J
C:\Users\Tcs>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet:
Connection-specific DNS Suffix . :
Link-local IPv6 Address . . . . . : fe80::d8a1:e21b:7bd7:849c%11
IPv4 Address. . . . . : 172.16.10.80
Subnet Mask . . . . . : 255.255.252.0
Default Gateway . . . . . : 172.16.8.1

```
C:\Users\Tcs>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    127.0.0.1:49682        DESKTOP-G0LVE4J:49683  ESTABLISHED
  TCP    127.0.0.1:49683        DESKTOP-G0LVE4J:49682  ESTABLISHED
  TCP    127.0.0.1:49684        DESKTOP-G0LVE4J:49685  ESTABLISHED
  TCP    127.0.0.1:49685        DESKTOP-G0LVE4J:49684  ESTABLISHED
  TCP    172.16.10.80:53081     4.213.25.240:https     ESTABLISHED
  TCP    172.16.10.80:53471     4.213.25.240:https     ESTABLISHED
  TCP    172.16.10.80:53476     a23-202-229-22:https   ESTABLISHED
  TCP    172.16.10.80:53477     57.155.48.216:https    ESTABLISHED
  TCP    172.16.10.80:53478     204.79.197.222:https   ESTABLISHED
  TCP    172.16.10.80:53480     150.171.110.84:https   ESTABLISHED
  TCP    172.16.10.80:53481     172.178.232.44:https   ESTABLISHED
  TCP    172.16.10.80:53482     172.16.7.152:ms-do     SYN_SENT
  TCP    [fe80::d8a1:e21b:7bd7:849c%11]:1521  DESKTOP-G0LVE4J:49680  ESTABLISHED
  TCP    [fe80::d8a1:e21b:7bd7:849c%11]:49680  DESKTOP-G0LVE4J:1521  ESTABLISHED
```

Result :-

Thus the network commands has been studied and executed successfully.

Date:- 28/7/26
Exp no:- 3

packet tracer tool installation and user interface

Aim:-

To study the packet tracer tool installation and user interface overview.

Procedure:-

1.) From the network component box, click and drag and drop the below components:

   a.) 4 Generic PCs and one HUB
   b.) 4 Generic PCs and one switch

2.) click on connections:

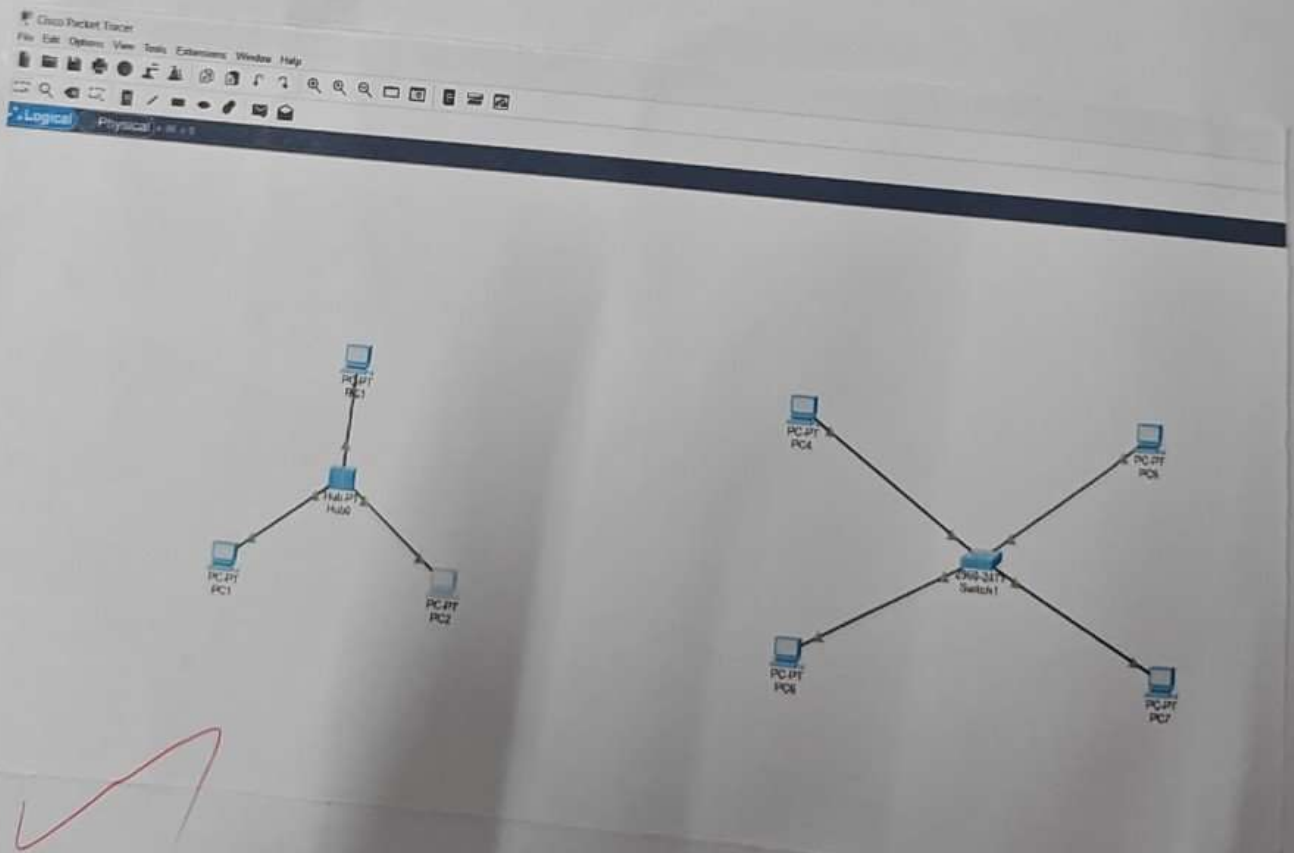
   a.) click on copper straight through cable
   b.) select one of the pc and connect it to HUB using the cable. The LINK LED should glow in green, indicating that the link is up.

3.) click on the PCs connected to the Hub, go to the Desktop tab, click on IP configuration and enter an IP address and subset mask.

click on the PDU (message icon) from the common tool bar.

a.) Drag & Drop it on one of PC and then drop it on another PC (destination machine) connected to the HUB




Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical
Physical
PC-PT PC1
Hub-PT Hub0
PC-PT PC1
PC-PT PC2
PC-PT PC4
PC-PT PC5
2960-24TT Switch1
PC-PT PC6
PC-PT PC7

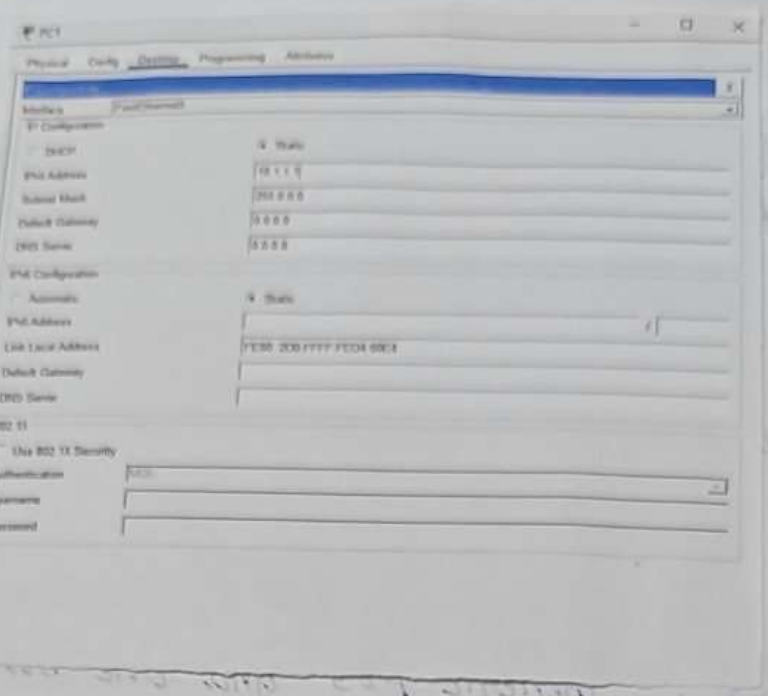
4.) observe the flow of PDU from source PC to destination pc by selecting the Realtime mode of simulation.

5.) Repeat step 3 to step 5 for the PCs connected to the switch

6.) observe how HUB & switch are forwarding the PDU and write your observation and conclusion about the behaviors of switch & HUB.

Result :-

Thus the study of packet tracer tool installation and user interface overview was successfully implemented.



PC1

Physical Config Desktop Programming Attributes

Interface FastEthernet0

IP Configuration

DHCP Static

IPv4 Address 10.1.1.1

Subnet Mask 255.0.0.0

Default Gateway 0.0.0.0

DNS Server 0.0.0.0

IPv6 Configuration

Automatic Static

IPv6 Address

Link Local Address FE80::200:FFFF:FED4:6BC4

Default Gateway

DNS Server

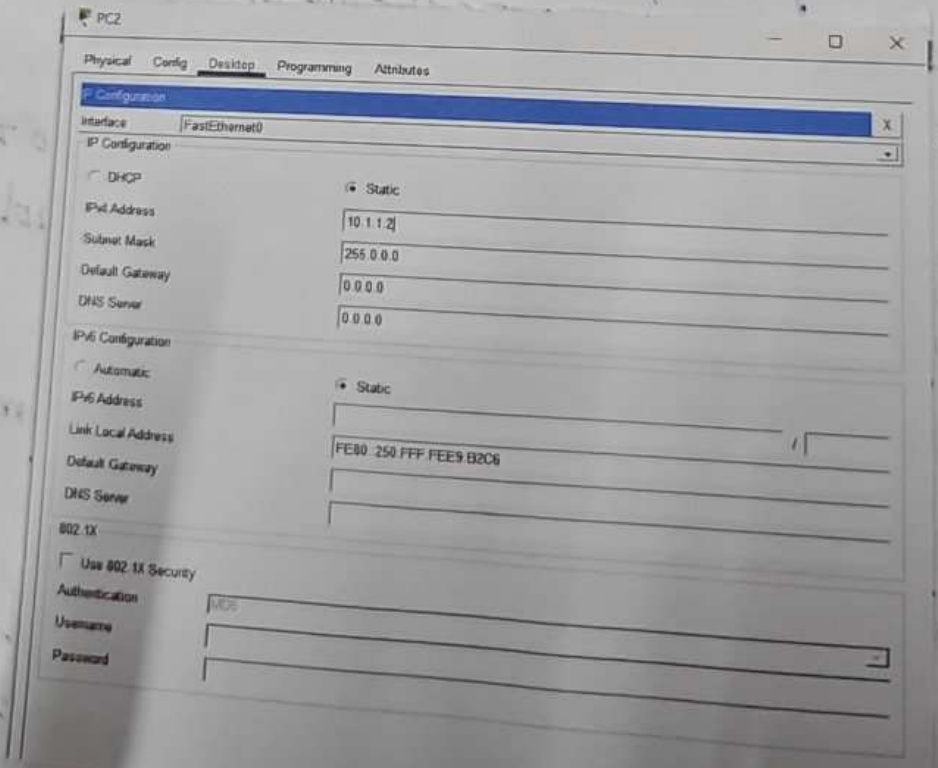
802.1X

Use 802.1X Security

Authentication MD5

Username

Password



PC2

Physical Config Desktop Programming Attributes

IP Configuration

Interface FastEthernet0

IP Configuration

DHCP Static

IPv4 Address 10.1.1.2

Subnet Mask 255.0.0.0

Default Gateway 0.0.0.0

DNS Server 0.0.0.0

IPv6 Configuration

Automatic Static

IPv6 Address

Link Local Address FE80::250:FFF:FEE9:B2C6

Default Gateway

DNS Server

802.1X

Use 802.1X Security

Authentication MD5

Username

Password

Date :- 2/8/26
Expno :- 2

study of Network cables

Aim :-

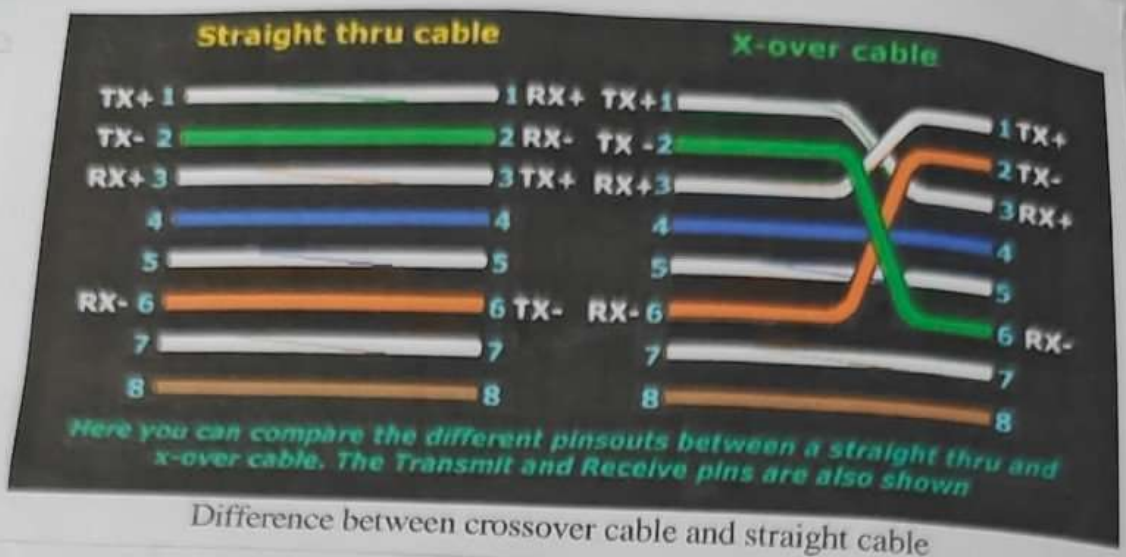
study of different types of Network cables

a) understand different types of network cable

Different type of cables used in networking are:

1. unshielded twisted pair (UTP) cable
2. shielded twisted pair (STP) cable
3. coaxial cable
4. Fibre optic cable

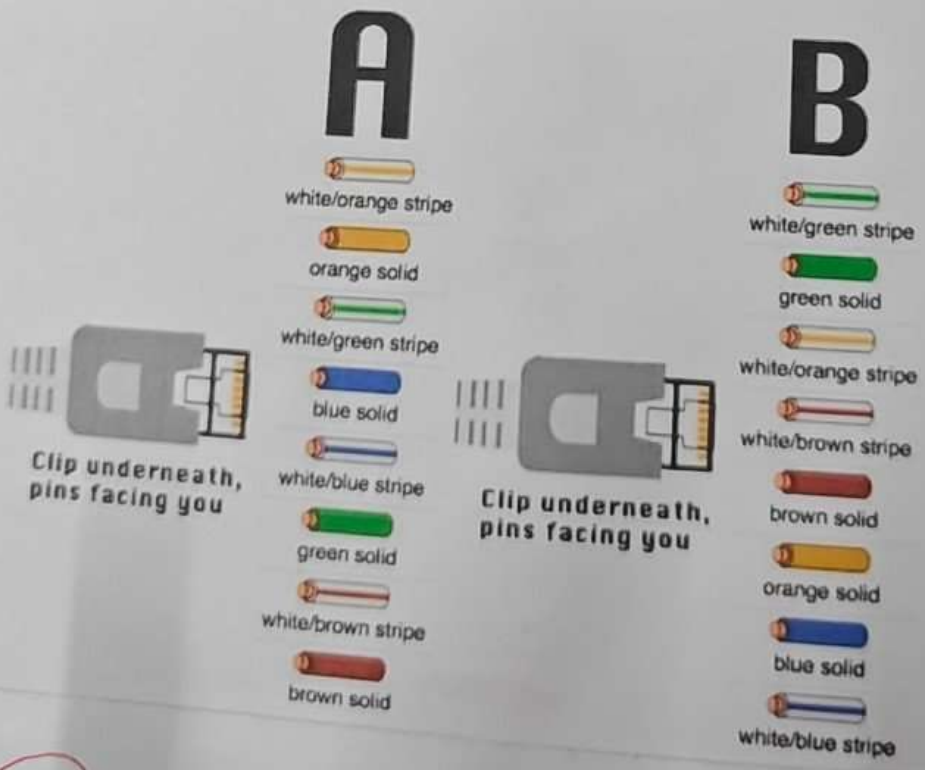
cable type	category	Maximum Data Transmission	Advantages / Disadvantages	Application / use
UTP	category 3	10 bps	Advantages * cheaper in cost * easy to install as they have a smaller overall diameter. Disadvantages * More prone to (EMI) Electromagnetic interference and noise	10 Base T Ethernet
	category 5	up to 100 mbps		Fast Ethernet, Gigabit Ethernet
	category 5e	1Gbps		Fast Ethernet, Gigabit Ethernet
STP	category 6, 6a	10Gbps	Advantages * shielded * Faster than UTP * less susceptible to noise and interference Disadvantages * Expensive * Greater installation effort	Gigabit Ethernet, 10G Ethernet (55m) widely used in data centres
SSTP	category 7	10Gbps		Gigabit ethernet 10G Ethernet (100m)



Difference between crossover cable and straight cable

int out the diagram below or have it handy as a reference

Straight through network cable: both sides should be A
Crossover cable: One side A, one side B



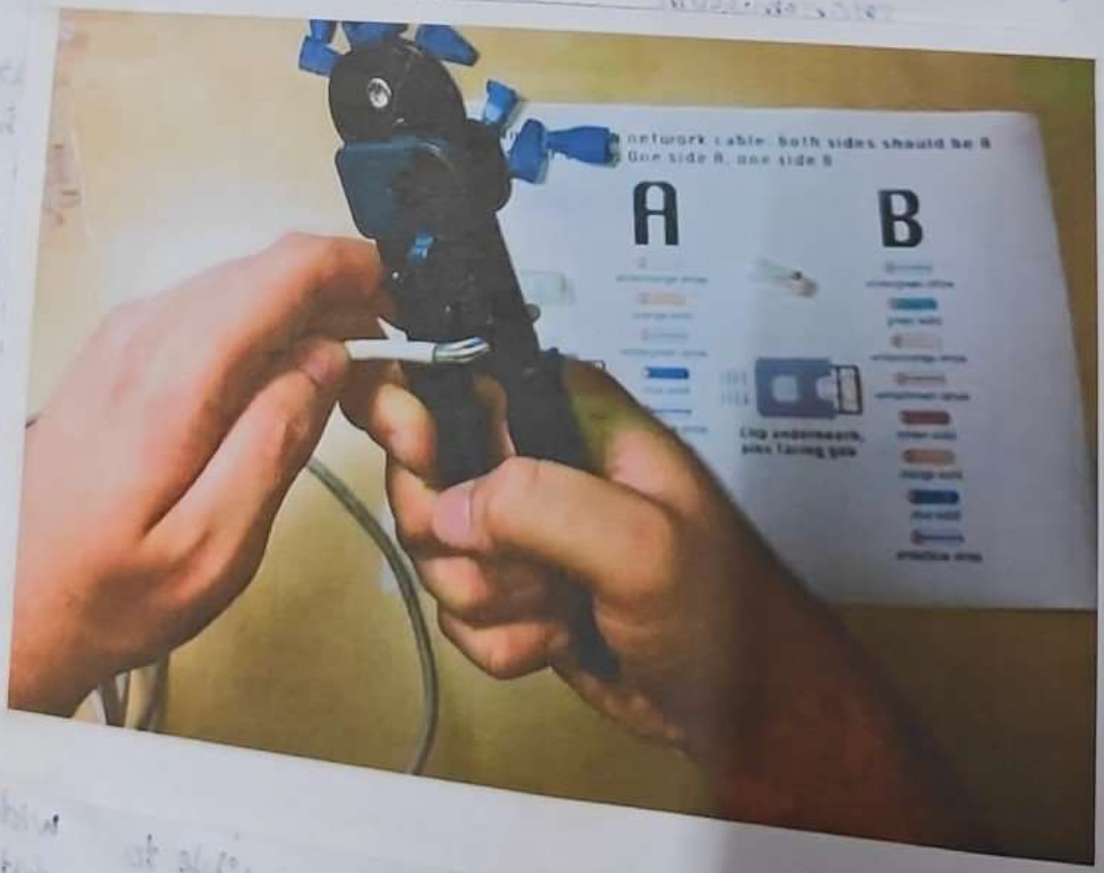
Coaxial cable	RG1 -6 RG1 -59 GG1 -11	10-100 Mbps	Advantages * High bandwidth * Immune to interfale * Low loss bandwith * versatile Disadvantages * Limited distanle * Lost * size is bulky	Speed of signal is 500m Telvision network High speed internet connections
fibre optics cable	single mode multi mode	100 Gbps	Advantages * High speed * High bandwidth * High security * Long distanle Disadvantages * Expensive * Requires skilled installers	Maximum distanle of fibre optics cable is around 100 meters

b) Make your own Ethernet cross-over cable /straight cable

Tools and parts needed:

* Ethernet cabling. CAT5e is certified for gigabit support, but CAT5 cabling works as well, just over shorter distanes.
* A crimping tool. This is an all in one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables.
* Two RJ 45 plugs
* optional two plug shields.


Clip underneath, pins facing you
white/blue stripe
green solid
white/brown stripe
Clip underneath, pins facing you


A
B

step 1:- To start construction of the device, begin by threading shields onto the cable.

step 2:- Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

step 3:- After, you will need to untangle the wires; there should be four "twisted pairs". Referencing back to the sheet, arrange them from top to bottom. one end should be in arrangement A and the other in B.

step 4:- once, the order is correct, bunch them together in a line, and if there are any that stick out farther than others, snip them back to create an even level. The different aspect is placing these into the RJ45 plug without messing up the order.

step 5:- Next, push the cable right in, The notch at the end of the plug needs to be just over the cable shielding and if it isn't, that means that you strippeed off too much sheilding. simply snip the cables back a little more.

step 6:- After the wires are securely sitting insiole the plug, insert it into the crimping and push down

step 7:- Lastly, repeat for the other end using diagram B/ using diagram A


T/A
orange solid
white/green stripe
blue solid
white/blue str
neath,


white/green stripe
green solid
white/orange stripe
white/brown stripe

Result :-

Thus, the study experiment has been successfully studied.


Cros
A
white/orange stripe
orange solid
white/green stripe
blue solid
white/blue stripe
underneath,
acing you

Exp no: 4
Date :- 6/8/26

study of Local Area Network using a switch & Ethernet cables

Aim :-

To set up and configure a local area network using a switch and ethernet cables and verify communication between connected computer.

Procedure :-

plan the network by arranging 4 computer, one network switch and ethernet cable.

connect each computer to the switch using ethernet cables.

open control panel → network → Internet → Network computer.

Right click Ethernet → properties → Tcp|IvPc → properties.

Select use the following ip address and assign

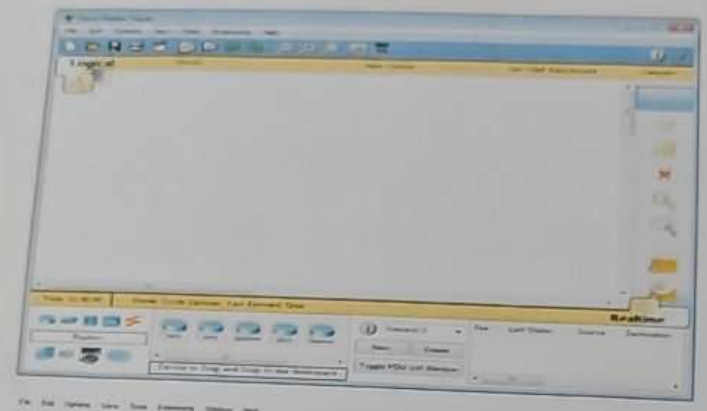
PC1 : 10 : 1.1.1, submit mask : 2250.0.0

PC2 : 10 : 1.1.2, submit mark : 255.0.0.0

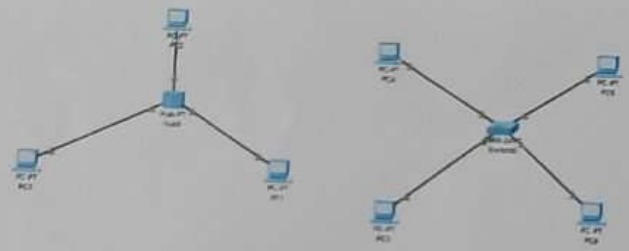
PC3 : 10 : 1.1.3, submet mark : 255.0.0.0

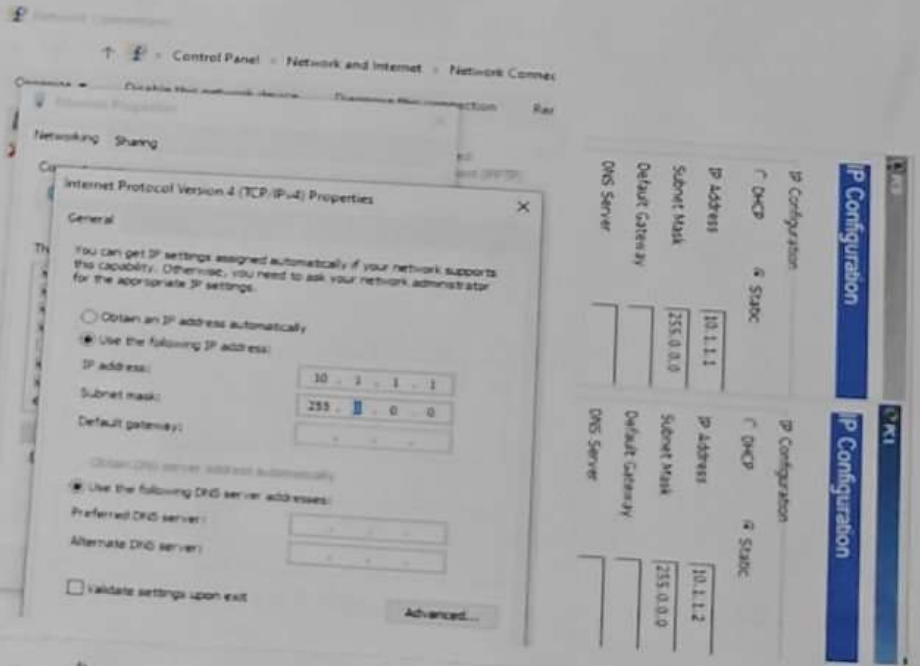
PC4 : 10 : 1.1.4, submet mark : 255.0.0.0

configure the switch by connecting pt to a computer, accessing gt, web interfaca and assigning the switch ip as 10.1.1.5 with 255.0.0.0.








Control Panel
Network and Internet
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address:
Subnet mask:
Default gateway:
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.1
255.0.0.0
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.2
255.0.0.0

Result :-

Thus, a local area network (LAN) was successfully setup and configured using a network switch and ethernet cables.

Exp no: 5
Date: 7/8/26

packet capture using wire shark

Aim :-

To capture filter inspect and analyze network packets using the wireshark packet Analyzer.

Algorithm :-

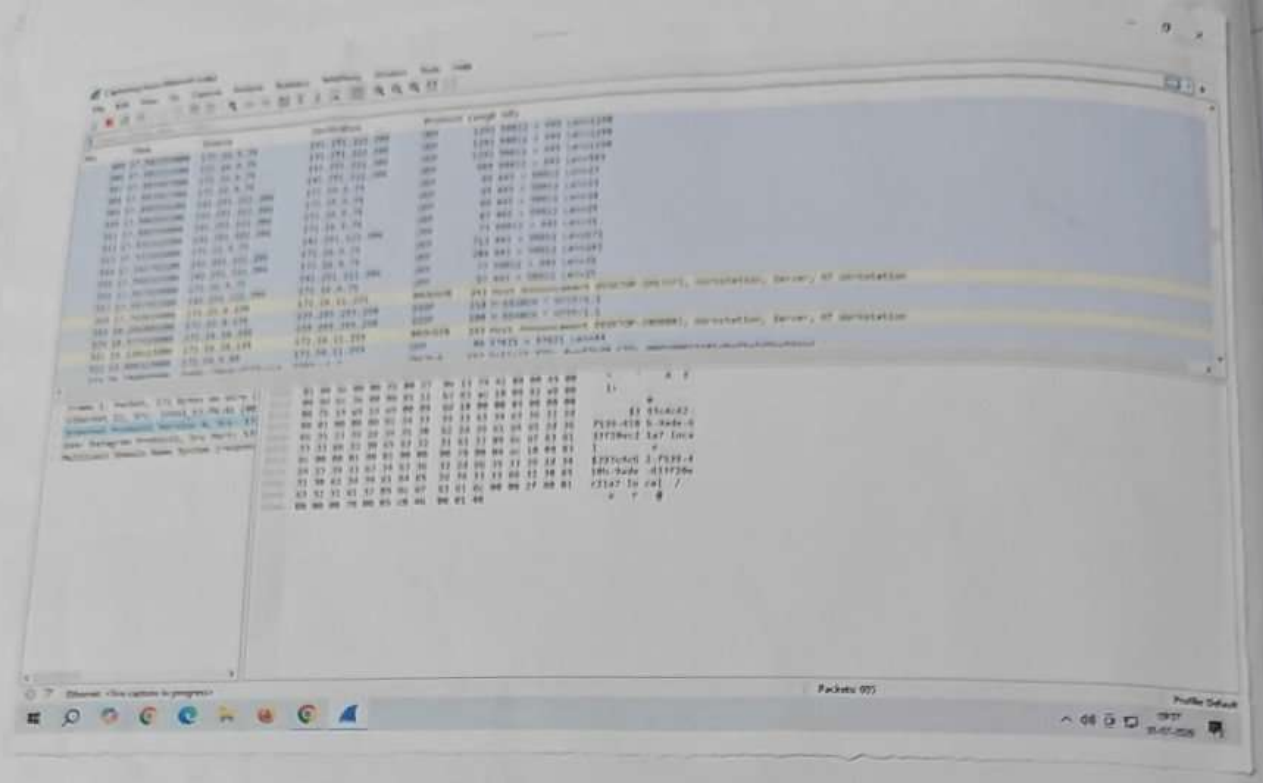
* open wireshark
* select the active network interface
* configure capture to stop after 100 packets
* start packet capture.
* Apply display filters
* Inspect packet details and view the flow graph
* saved the captured packets

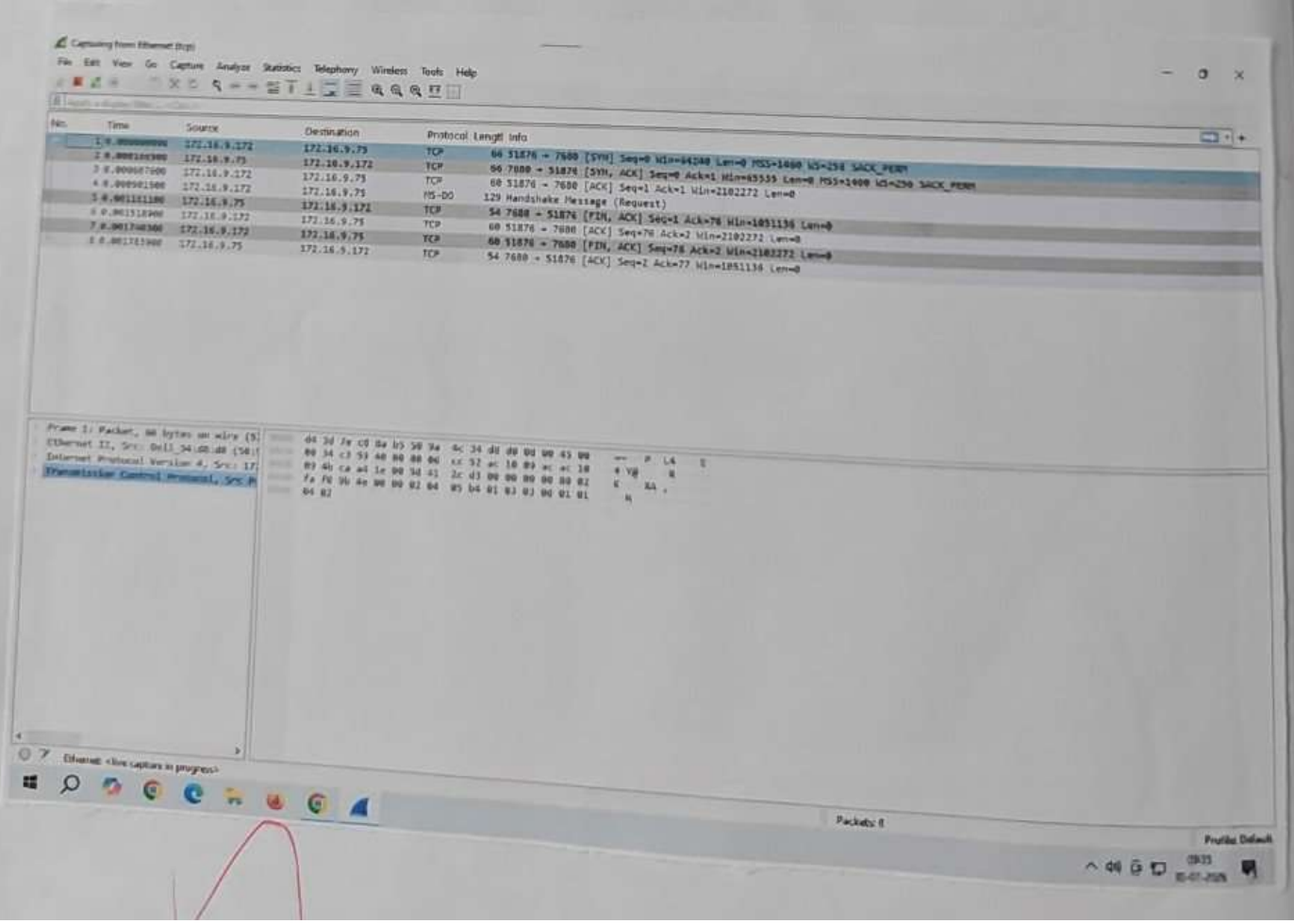
procedure :-

* Launch wireshark and subset the active network interface
* Go to capture → options and set stop capture after 10 packets.
* click start to design packet capture

TCP / UDP : tcp || udp
ARP : arp
DNS : dns
HTTP : http
HDCP : bootp

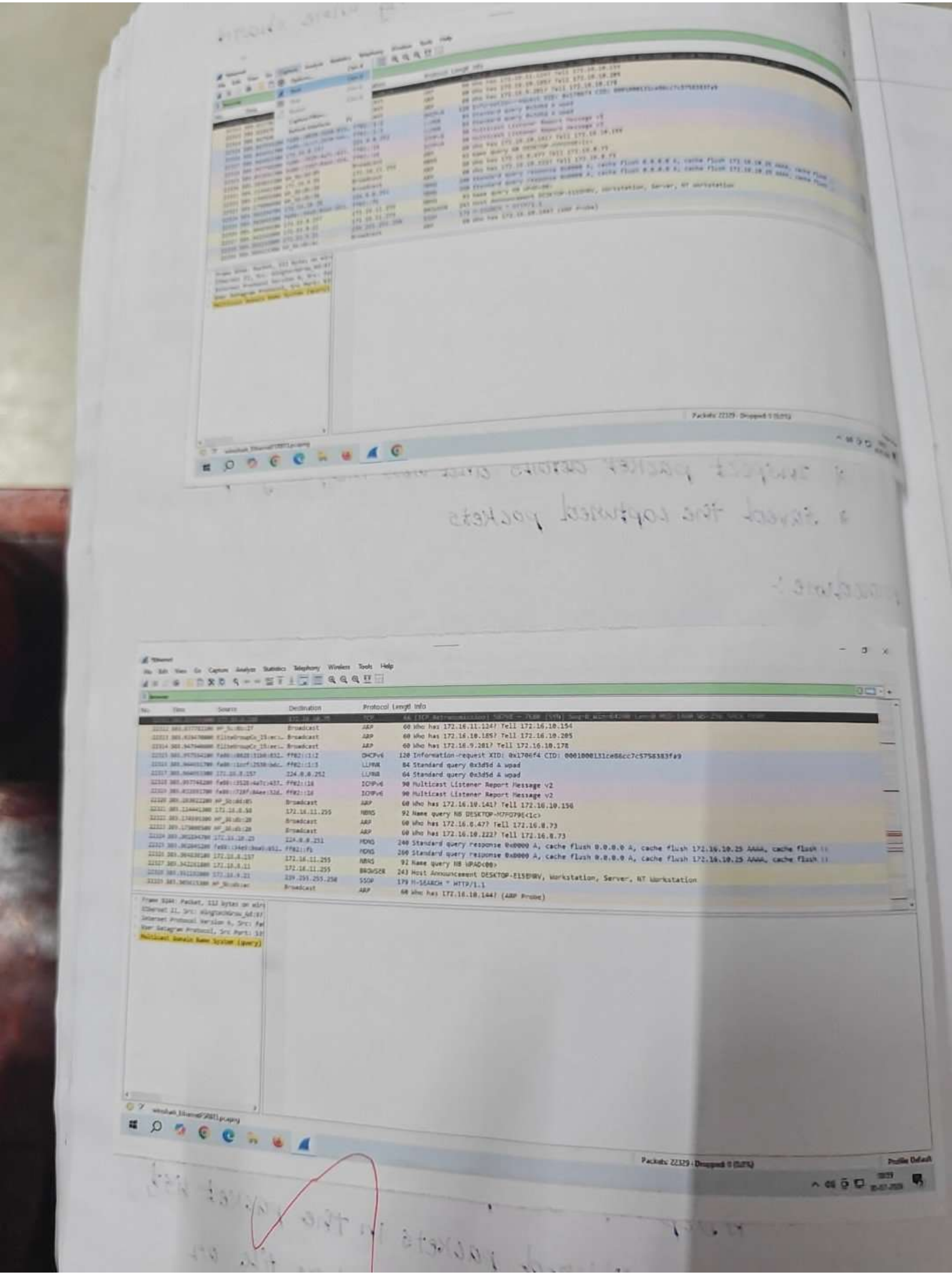
Inspect the filtered packets in the packet list, details and bytes panes. save the capture file or future analysis.




Capturing from Ethernet (tcp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
1 0.000000000 172.16.9.172 172.16.9.75 TCP 66 51876 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2 0.000169900 172.16.9.75 172.16.9.172 TCP 66 7680 → 51876 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 WS=256 SACK_PERM
3 0.000607600 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=1 Ack=1 Win=2102272 Len=0
4 0.000901500 172.16.9.172 172.16.9.75 MS-DO 129 Handshake Message (Request)
5 0.001161100 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [FIN, ACK] Seq=1 Ack=76 Win=1051136 Len=0
6 0.001518900 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=76 Ack=2 Win=2102272 Len=0
7 0.001748300 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [FIN, ACK] Seq=76 Ack=2 Win=2102272 Len=0
8 0.001783900 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [ACK] Seq=2 Ack=77 Win=1051136 Len=0
Ethernet: <live capture in progress>
Packets: 8
Profile: Default

Result :-

successfully captured 100 network packets.
applied display filters for TCP, UDP, ARP, DNS, HTTP, IP, ICMP, DHCP analyzed packets details using wireshark.


Source
Destination
Protocol Length Info
ARP 60 Who has 172.16.11.124? Tell 172.16.10.154
ARP 60 Who has 172.16.10.185? Tell 172.16.10.205
ARP 60 Who has 172.16.9.201? Tell 172.16.10.178
DHCPv6 120 Information-request XID: 0x1706f4 CID: 000100013...
LLMNR 84 Standard query 0x3d5d A wpad
LLMNR 64 Standard query 0x3d5d A wpad
ICMPv6 90 Multicast Listener Report Message v2
ICMPv6 90 Multicast Listener Report Message v2
ARP 60 Who has 172.16.10.141? Tell 172.16.10.156
NBNS 92 Name query NB DESKTOP-M7FO79E<1c>
ARP 60 Who has 172.16.0.47? Tell 172.16.8.73
ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
MDNS 240 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush
MDNS 260 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush
NBNS 92 Name query NB WPAD<00>
BROWSER 243 Host Announcement DESKTOP-E15EMBV, Workstation, Server, NT Workstation
SSDP 179 M-SEARCH * HTTP/1.1
ARP 60 Who has 172.16.10.144? (ARP Probe)
Packets: 22329 · Dropped: 0 (0.0%)
Profile: Default

Expno :- 6
Date :- 6/8/26

Error Detection and correction using Hamming code

Aim :-

To write a program to implement error detection and correction using the Hamming code.

Algorithm :-

1) Start the program
2) Read the number of 4-bit data blocks (n) forming the input data stream.
3) For each 4-bit data block D = d1 d2 d3 d4, perform Hamming encoding:
   a) Arrange 7 bits with parity bits at positions 1, 2 and 4 and data bits at positions 3, 5, 6 and 7.
   b) compute the parity bits using even parity:
      P1 = d1 XOR d2 XOR d4 (covers bit positions 1, 3, 5, 7)
      P2 = d1 XOR d3 XOR d4 (covers bit positions 2, 3, 6, 7)
      P4 = d2 XOR d3 XOR d4 (covers bit positions 4, 5, 6, 7)
   c) Form the 7-bit Hamming code word : P1 P2 d1 P4 d2 d3 d4
4) simulate transmission by introducing a single-bit error at a chosen bit position in the code word.
5) At the receiver, recompute the check bits from the received code word.
6) compute Error position = C4 C2 C1
7) If Error position = 0, no error has occurred.

8.) Extract the original 4 data bits from the corrected 7-bit code word.

9.) Repeat steps 3 to 8 for all n blocks in the data stream.

10.) Stop the program.

program:

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4] -> returns 7-bit
hamming code list
        positions (1-indexed): 1=p1 2=p2 3=d1 4=p4 5=d2
6=d3 7=d4 """
h = [0] * 7
h[2] = d[0]  # position 3
h[4] = d[1]  # position 5
h[5] = d[2]  # position 6
h[6] = d[3]  # position 7

h[0] = h[2] ^ h[4] ^ h[6]
h[1] = h[2] ^ h[5] ^ h[6]
h[3] = h[4] ^ h[5] ^ h[6]

return h

def detect-and-correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
```

Output :

Enter number of 4-bit data blocks in the stream : 2

--- Block 1 ---

Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1

original Data Bits : 1011

Encoded Hamming code (7 bits) : 0 1 1 0 0 1 1

Enter bit position to introduce error (1-7, 0 = no error): 3

Transmitted code (with error induced at pos 3): 0 1 0 0 0 1 1

Relieved code : 0 1 0 0 0 1 1

Checking for errors...

Error detected at bit position : 3 → correcting

corrected code : 0 1 1 0 0 1 1

Recovered Data Bits : 1 0 1 1

--- Block 2 ---

Enter 4 data bits (space seperated e.g. 1 0 1 1): 0 1 1 0

original Data bits : 0 1 1 0

Encoded Hamming code (7 bits)

```
    error_pos = c1*1 + c2*2 + c4*4

    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos}
                                    -> correcting...")
        h[error_pos - 1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=== HAMMING CODE - ERROR DETECTION AND
                                    CORRECTION ===\n")
    n = int(input("Enter number of 4-bit data blocks in the
    stream: "))

    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits (space
                                    separated, e.g. 1 0 1 1): ").split()))

        h = encode(d)
        print(f"Original Data Bits  : {to_str(d)}")
        print(f"Encoded Hamming Code (7 bits): {to_str(h)}")

        pos = int(input("Enter bit position to introduce error
                                    (1-7, 0 = no error): "))
```

Date :- 10/8/26
Exp no:- 7

Flow control of Data Link Layer using sliding window protocol

Aim :-

To write a program to implement flow control at the data link layer using the sliding window protocol.

Algorithm :-

1.) start the program.

2.) Read the total no of frames to be sent (N) the sender's window size (w)

3.) Initialize base = 0

4.) Repeat while base < N

a.) Determine the current sender window as the range

b.) send all the frames with sequence numbers from next-seq

c.) If the last frame lies within the frames just sent

i.) The receiver discards the sent frame and any subsequention

ii.) The sender times out and retransmits all frames.

d.

i.) The receiver accepts each frame in order and sends a kumulative acknowledgment.

Sender window: [frame 4 – frame 7]

Node A → sending frame 5

Node A → sending frame 6

Node A → sending frame 7

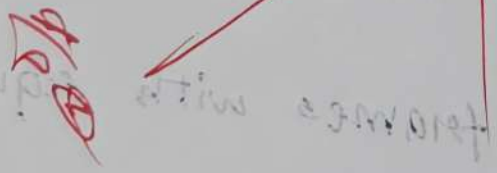
Node B → Received frame 4, sending ACK 5

Node B → received frame 5, sending ACK 6

Node B → received frame 7, sending ACK 8

Node A → ACK 7 received, sliding window forward

All 8 frames successfully transmitted from node A to node B.



ii.) On receiving the Ack for the last frame sent, the sender slides the window forward.

5) Repeat steps until all N frames transmitted.

6) Stop

Program:-

```
def simulate(total_frames, window_size, ack_frame):
    base = 0
    next_seq = 0
    while base < total_frames
    window_end = min(base + window_size - 1, total_
                                    frames-1)
    print(f"sender window: frame {base} - frame
                                    {window_end}]")
while next_seq < base + window_size and next_seq <
                                    total_frames:
    if next_seq == lost_frame
        print(f"{frame {next_step} LOST in
                                    transit:))
    next_seq = 1
if lost_frame is not none and base <= lost_frame <
                                    next_eq:
    print(f"Node B --> frame {Lost_frame} missing.
                    discarding out of order frames after it.")
    Next_seq = Lost frame
    Lost_frame = None
for f in range(base, next_seq):
```

```
print(f"Node A --> ACK {next_seq - 1} received. sliding window forward :)")

def main():
    total_frames = int(input("Enter total no of frames to send :"))
    lost_frame = int(input())
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)

if __name__ == "__main__":
    main()
```

Result :-

The program to implement flow control at the data link layer using the sliding window protocol was successfully executed and verified by simulating the flow of frames between two nodes.

Expno: 9
Date: 24/9/26

Implementation of subnetting in cisco packet tracer

Aim :-

To implement classess ip subnetting using the cisco packet tracer simulator and test connectivity between different subnets.

Requirements :-

cisco packet tracer
Routers
switchers
pcs
Ethernet cables

Procedure :-

1.) creating the network topology

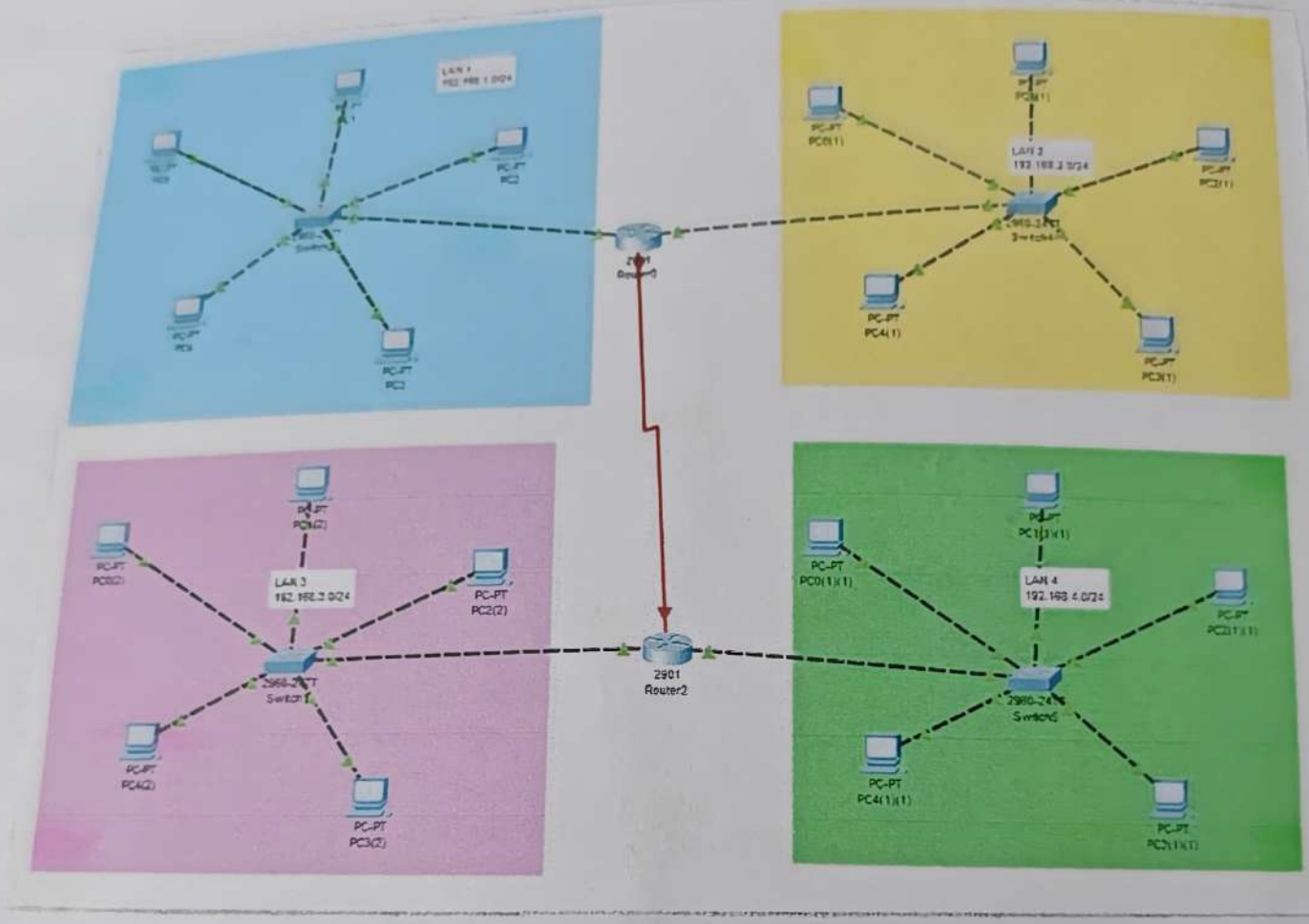
open cisco packet tracer
create a new workspace
Add the required routers, switches and pcs
connect the device using appropriate Ethernet cable.

2.) ip Addressing :

LAN1	R1G0/0	192.168.1.1/27
LAN2	R1G0/1	192.168.2.1/27
LAN3	R2F0/0	192.168.3.1/27
LAN4	R2F0/1	192.168.4.1/27

For example;

IP Address : 192.168.1.11
Subnet mask : 255.255.255.224
Default gateway : 192.168.1.1


LAN 1
PC-PT
PC-PT
PC-PT
PC-PT
PC-PT
LAN 2
192.168.2.0/24
PC-PT PC0(1)
PC-PT PC1(1)
PC-PT PC2(1)
2960-24TT Switch4
PC-PT PC4(1)
PC-PT PC3(1)
LAN 3
192.168.3.0/24
PC-PT PC0(2)
PC-PT PC1(2)
PC-PT PC2(2)
2960-24TT Switch3
PC-PT PC4(2)
PC-PT PC3(2)
2901 Router2
LAN 4
192.168.4.0/24
PC-PT PC0(1)(1)
PC-PT PC1(1)(1)
PC-PT PC2(1)(1)
2960-24TT Switch5
PC-PT PC4(1)(1)
PC-PT PC3(1)(1)

3. configuring Router R1

open R1 → CH and enter:

enable

configure terminal

interface gigabit Ethernet 0/10

ip address 192.168.1.1.255.255.254

no shut down

exit

4- configuring Router R2

enable

configure terminal

interface fast ethernet 0/10

ip address 192.168.31.255.255.224

no shut down

exist

5. configuring the ethernet:

For S1:

enable

configure terminal

interface fast Ethernet 0/1

switch port mode access

Exit

6. configuring the pcs:

open each pc → Desktop → ip configuration and enter the appropriate ip address subnet mask and default gateway.

PC	IP address	Subnet mask	Default gateway
PC1	192.168.1.11	255.255.255.244	192.168.1.1
CP2	192.168.1.12	255.255.255.244	192.168.1.1

Testing the network:

1. Open PC → Desktop → command prompt
   ping 192.168.1.1
2. Test connectivity b/w pcs in the subnet : 192.168.1.12

Result :-

Implemented classess IP subnetting using the cisco packet tracer simulator is successfully completed.

Exp no: 10(a) a) Internetworking using Routers
Date :- 30/8/26

Aim :-

To design and configure a simple internetwork using a router and two in cisco packet tracer and test connectivity using ppu.

Step 1 :- configure Router 1

1) open cisco packet tracer
2) select Router → Router 1
3) open w 1 and press enter
4) enter
   enable
   config t
5) configure fact Ethernet 0/0:
   Interface fast Ethernet 0/0
   ip address 192.168.10.1255.255.255.0
   No shut down
6) configure fast ethernet d0:
   Interface fas Ethernet 0/1
   ip address 192.168.101.255.255.255.0.
   NO shutdown

Step 2 :- configure pc0

Go to pc0 → desktop → Ip configwiation
ip address : 192.168.10.2
Subnet Host : 255.255.255.0
Default Gateway : 192.168.10.1





Step 3 : configure pc1

Go to pc1 → desktop → IP configuration
IP Address : 192.168.20.2
subnet mask : 255.255.255.0
Default Gateway : 198.168.20.1

Step 4 :· connect the devices

use copper straight - through cable
PC 0 Fast ethernet 0 → Router 1 Fast ethernet 0/0
PC1 Fast ethernet 0 → Router 1 Fast ethernet 0/1

Step 5 :· Test connectivity

1.) select simulation mode
2.) choose add simple PDU
3.) click pc0 → pc1
4.) If the PDU is successful, the two network are communicating through the router.

Result :-

The internetwork was successfully designed and configured, and connectivity between pc0 and pc1 was verified.

Exp no: 11

Date: 17/9/26 Routing at Network layer

Aim:-

To simulate and configure static routing between multiple networks using cisco packet tracer, configure primary and backup static routers using administrative distance.

Requirements:-

cisco packet Tracer

three - routers - Router 0, Router 1, 2

PCs, end devices

Properly assigned Ip address.

observation:-

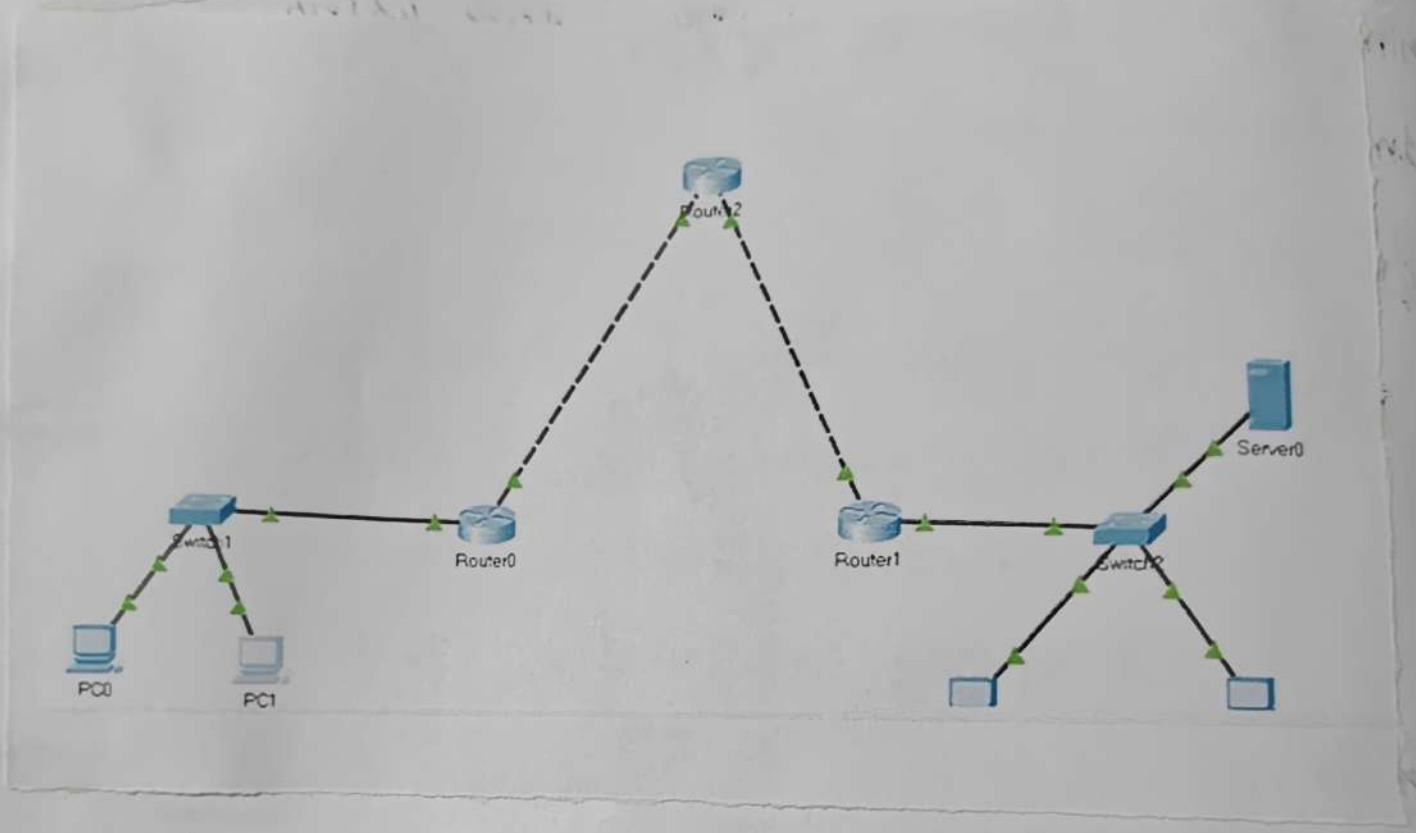
A network topology consisting of three routers, router 1, 2 was created in cisco packet tracer, different networks user connected and user assigned suitable Ip address.

Router 1:

Router 1 was configured with static routers to the 10.0.0.0/8 to 40.0.0.0/8 networks. the route through roughest 0 was assigned a lower administrative distance primary route the configuration was verified using show ip route static

Router 2:-

was configured with static router to the 10.0.0.0/8 & 30.0.0.0/8 networks. The routing table and verified


Router2
Server0
Switch1
Router0
Router1
Switch2
PC0
PC1

using show ip routers static

Backup route verification:

The main link between router & was disconnected to simulate a link failure connectivity was verified using link & traler.

Verification:

show ip router static. The routing tables showed the configured static routes & their administrative using the pin command.

Deleting a static router:

show ip route static. After deleting the Main route, the backup route becomes the active.

Result:-

Thus, static routing was successfully configured & simulated using cisco packet traler.

Expno: 10b
Date: 15/9/26

Wireless Router, HDCP server and Internet cloud

Aim:-

To design and configure and internetwork using a wireless. router, DHCP serves and Internet cloud and verify connectivity.

step 1: Build the Network

Add:

PC
wireless Router
Laptop
cable Modern
Internet cloud
cisco.com server

step 2: connect Devices

use the appropriate cables

PC → wireless router → cable modern:

copper straight - through

wireless Router → cable modern:

copper straight → through

cable Modern → Internet cloud

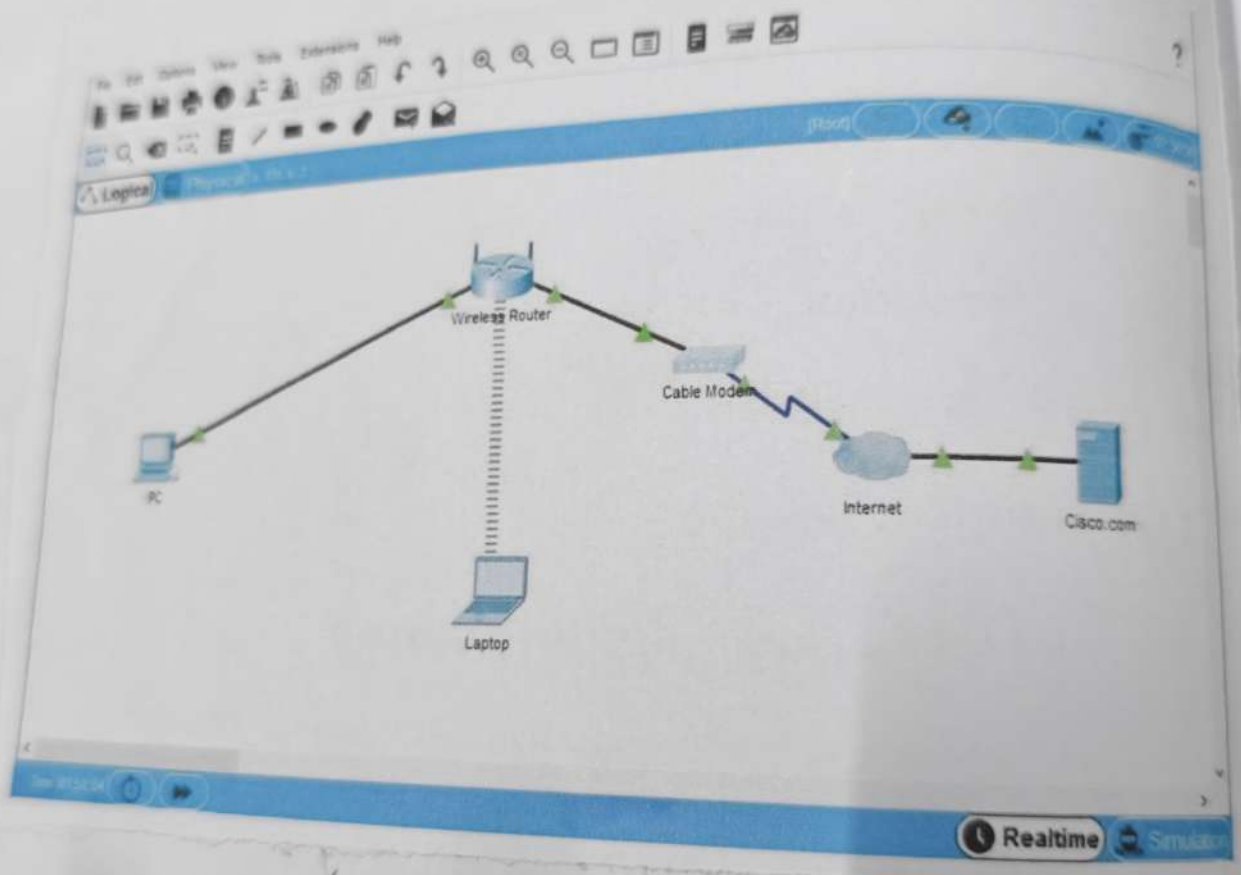
coaxial cable

Internet cloud → cisco.com server

copper straight - through

step 3: configure wireless router

1) open wireless Router → GUI
2) Go to wireless


Logical
Wireless Router
Cable Modem
PC
Internet
Cisco.com
Laptop
Realtime

4.) click save settings

step4:-

configure cisco.com server

DHCP

DHCP : ON

Pool Name : DHCP pool

default gateway : 208.67.220.220

starting ip : 208.67.220.1

Maximum users : 50

click add

step 5:- Test connectivity

1.) open pc → Desktop → command prompt

2.) Type : ipconfig /release

3.) Then : ipconfig /renew

4.) Test the server : ping cisco.com

Result :-

The wireless internetwork was successfully designed using a wireless router DHCP server and internet cloud and connectivity was verified using the ping comment

Exp 12

Part A: Echo Client-Server using TCP Sockets

AIM

To write and implement an Echo Client-Server program using TCP sockets in C.

Algorithm

Server Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Put the server socket in listening mode using listen().
4. Accept an incoming client connection using accept().
5. Read the message sent by the client using read() or recv().
6. Echo the same message back to the client using write() or send().
7. Close the client and server sockets using close().

Client Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Connect to the server's IP address and port number using connect().
3. Prompt the user for input and send the message to the server using write() or send().
4. Receive the echoed message back from the read() or recv().
5. Display the received message on the termin
6. Close the socket using close().

Code Implementation (TCP)

`tcp_server.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
    if ((server_fd = socket(AF_INET, SOCK_STR
        perror("Socket creation failed");
```

Result:

Thus, the echo client server was implemented using sockets.

Date:- 22/9/26
Expno:- 12

Implement echo client server using TCP/UDP sockets

Aim:-

To write and implement can echo client server program using TCP sockets inc.

Result:-

Thus, the echo client server was successfully implemented using sockets.

EXP 13

AIM

To implement a custom Ping program using raw ICMP sockets (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
   - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
   - Set Sequence Number and Process Identifier (PID) fields.
   - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
   - Wait for incoming packets with recvfrom() within a timeout window.
   - Parse the outer IP header to extract the inner ICMP packet.

26/9/26

PING1 program

   - Verify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the proc ID and sequence number.
6. **Calculate Metrics:** Subtract transmission timestamp receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time Live), packet size, and RTT. Repeat for desired numb pings.

Code Implementation (ping.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <sys/time.h>

#define PACKET_SIZE 64
#define PORT_NO 0
#define TIMEOUT 1 // Timeout in seconds
```

reachability of a host in calculated s

Exp no:- 13

PING program

Aim:-

To implement a ping program using python raw sockets to check the reachoffy of a clust and calculate the round trip time

Result:-

Thus ping program using python to check the reachability of a host in calculated successfully.

EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
   - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
   - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
   - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
   - **Extract Transport Header:**
     - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

     If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.

Clean Up: Close the socket upon receiving a termination signal.

ode Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int
  // Extract IP Header
  struct iphdr *ip_hdr = (struct iphdr *)buffer;
  unsigned int ip_hdr_len = ip_hdr->ihl * 4;

  struct sockaddr_in source, dest;
  memset(&source, 0, sizeof(source));
  source.sin_addr.s_addr = ip_hdr->saddr;

  memset(&dest, 0, sizeof(dest));
  dest.sin_addr.s_addr = ip_hdr->daddr;

  printf("\n================ Packet Captured
================\n");
```

Thus the network ... implemented successfully to capture TCP and UDP.

Exp no: 14
Date: 30/9/26

Network Packet sniffer using Raw sockets

Aim :-

To write and implement a network pocket sniffer using Raw sockets in c to capture and parse IP, TCP and UDP

Result :-

Thus the network sniffer using raw sockets is implemented successfully to capture and parse IP, TCP and UDP.

EXP 15

AIM

To analyze different types of web logs using the Webalizer log analysis tool and generate web traffic reports.

Procedure

Step 1: Run Webalizer

1. Download and extract the Webalizer executable (webalizer.exe) on Windows.
2. Place the executable in a dedicated working folder (e.g., C:\webalizer).

Step 2: Input Web Log File

1. Obtain or download a sample web server access log file (e.g., access.log from Apache, Nginx, or IIS).
2. Move the log file into the same directory as webalizer.exe or update the webalizer.conf configuration file to point to the input log location:

```
LogFile access.log
OutputDir ./output
```

Step 3: Run Webalizer

1. Open Command Prompt (cmd) and navigate to the Webalizer directory:

```
DOS
cd C:\webalizer
```

2. Execute Webalizer specifying the log file:

```
DOS
webalizer.exe access.log
```

3. Open the generated index.html file in any web browser to view the generated traffic report.

Exp no : 15
Date : 5/10/24

Types of web logs using webalizer Log Analysis

Aim :-

To analyze different types of web logs using the webalizer log analysis tool.

Result :-

Thus different types of web logs using the webalizer log analysis is analyzed sucessfully.

Completed

INDEX

NAME: Meeghasree.D STD.: _____ SEC.: AIDS ROLL NO.: 155 SUB.: computer networks observation

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	24/7/26	Network components used in Linux		
3	28/7/26	packet tracer tool installation and user interface		
2	2/8/26	study of network cables		
4	6/8/26	study of local area Network using a switch & Ethernet cables		
5	7/8/26	packet capture using wireshark		
6	8/8/26	error Detection and correction using Hamming code		
7	10/8/26	Flow control of Data Link layer using sliding window protocol		
8	24/8/26	Implementation of subnetting in cisco packet tracer		
10(a)	30/8/26	Internetworking using Routers		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10 (b)	15/9/26	Wireless Router, HDCP server and Internet cloud		
11	17/9/26	Routing at network layer		
12	22/9/26	Implement echo client server using TCP/UDP sockets		
13	26/9/26	PING program		
14	30/9/26	Network packet sniffer using Raw sockets		
15	5/10/26	Types of web logs using webalizer log analysis		

Completed